

Rețele de calculatoare

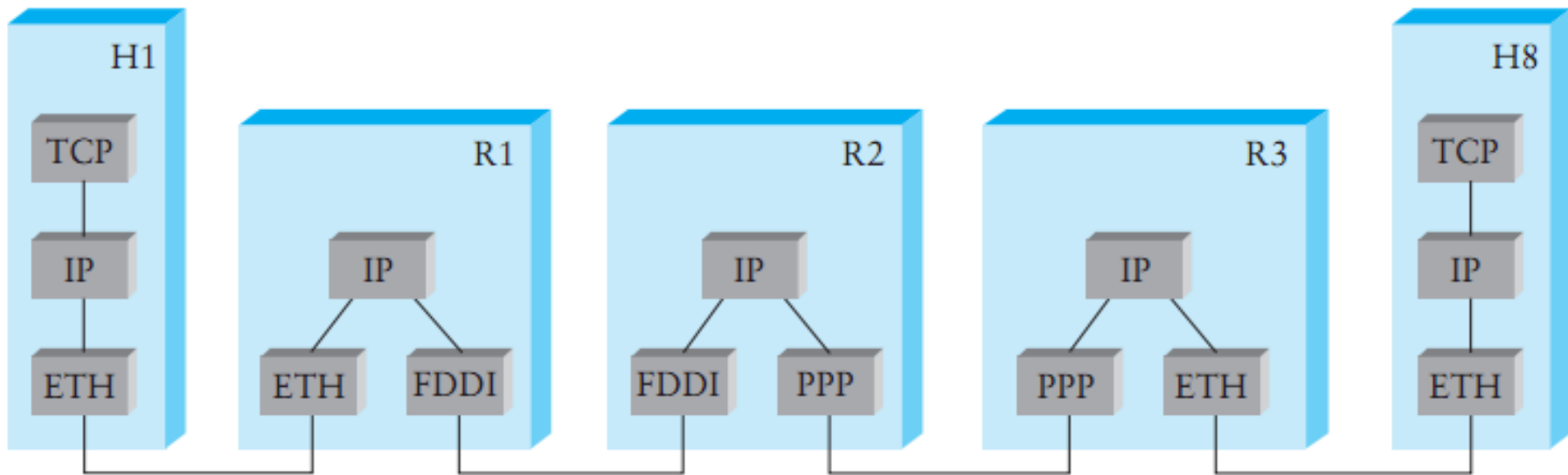
Partea a 5-a

Sebastian Fuicu

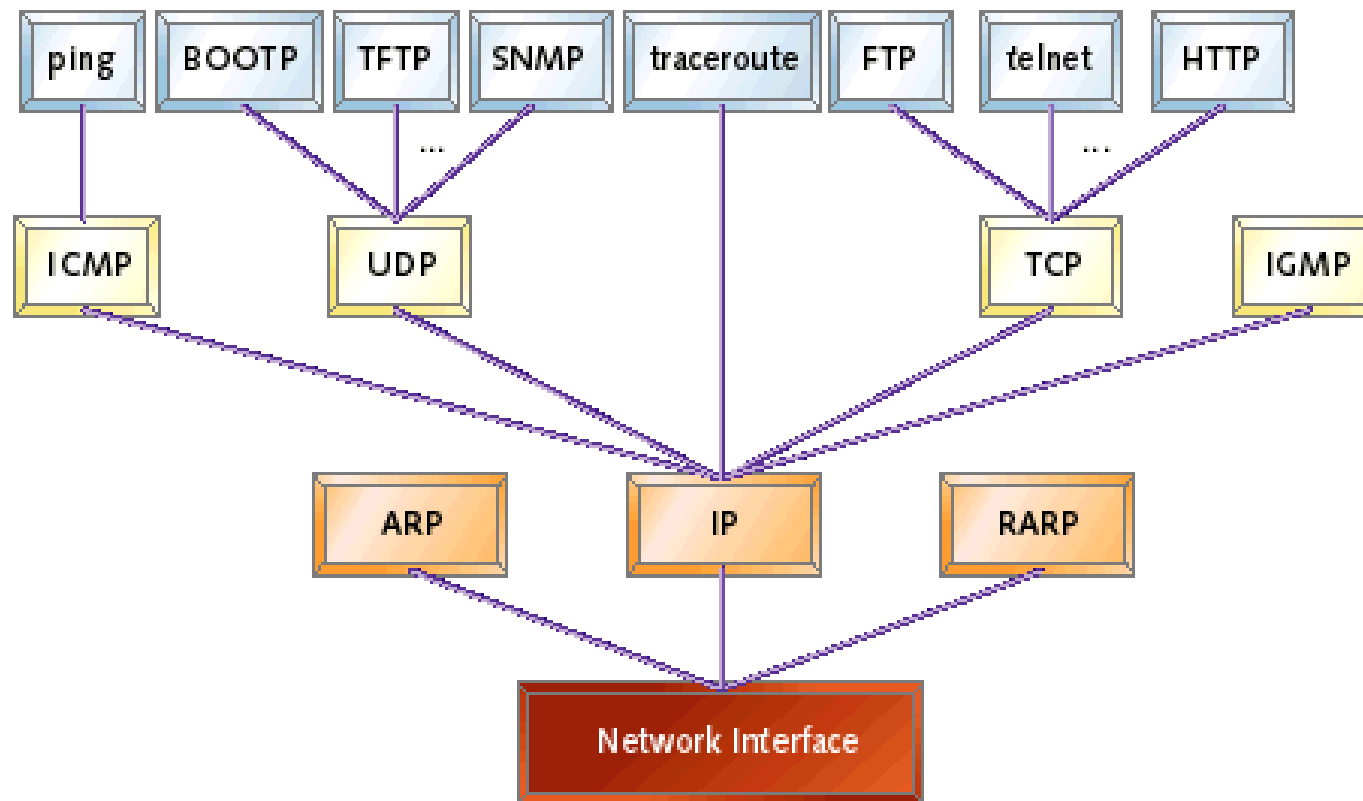
- **Interconectarea rețelelor**
- **Protocolul IP**
- **Protocolul TCP**
- **Controlul congestiei**

Interconectarea rețelelor

- Interconectarea rețelor reprezintă mecanismul prin care o colecție arbitrară de rețele oferă împreună un serviciu de transport de tip host-to-host.
- Protocolul IP este elementul cheie folosit pentru a construi rețele interconectate.
- O instanță a acestui protocol trebuie să ruleze pe fiecare din nodurile rețelelor interconectate.



Protocolul IP



Protocolul IP

Serviciul oferit de protocolul IP

- Modelul serviciului prezintă două componente:
 - o schemă de adresare care permite identificarea tuturor host-urilor interconectate.
 - un model de transfer al datelor, de tip datagramă, fără conexiune, model numit si „best effort”.
- Noțiunea de „best effort” se traduce prin aceea că dacă un pachet se pierde sau este afectat de eroare, rețeaua nu face nimic pentru a-l recupera, pentru ca ea a depus deja tot efortul pentru a transporta acel pachet.
- Este posibil ca pachetele care au fost transmise să ajungă în altă ordine decât cea în care au fost transmise sau unele să fie duplicate.

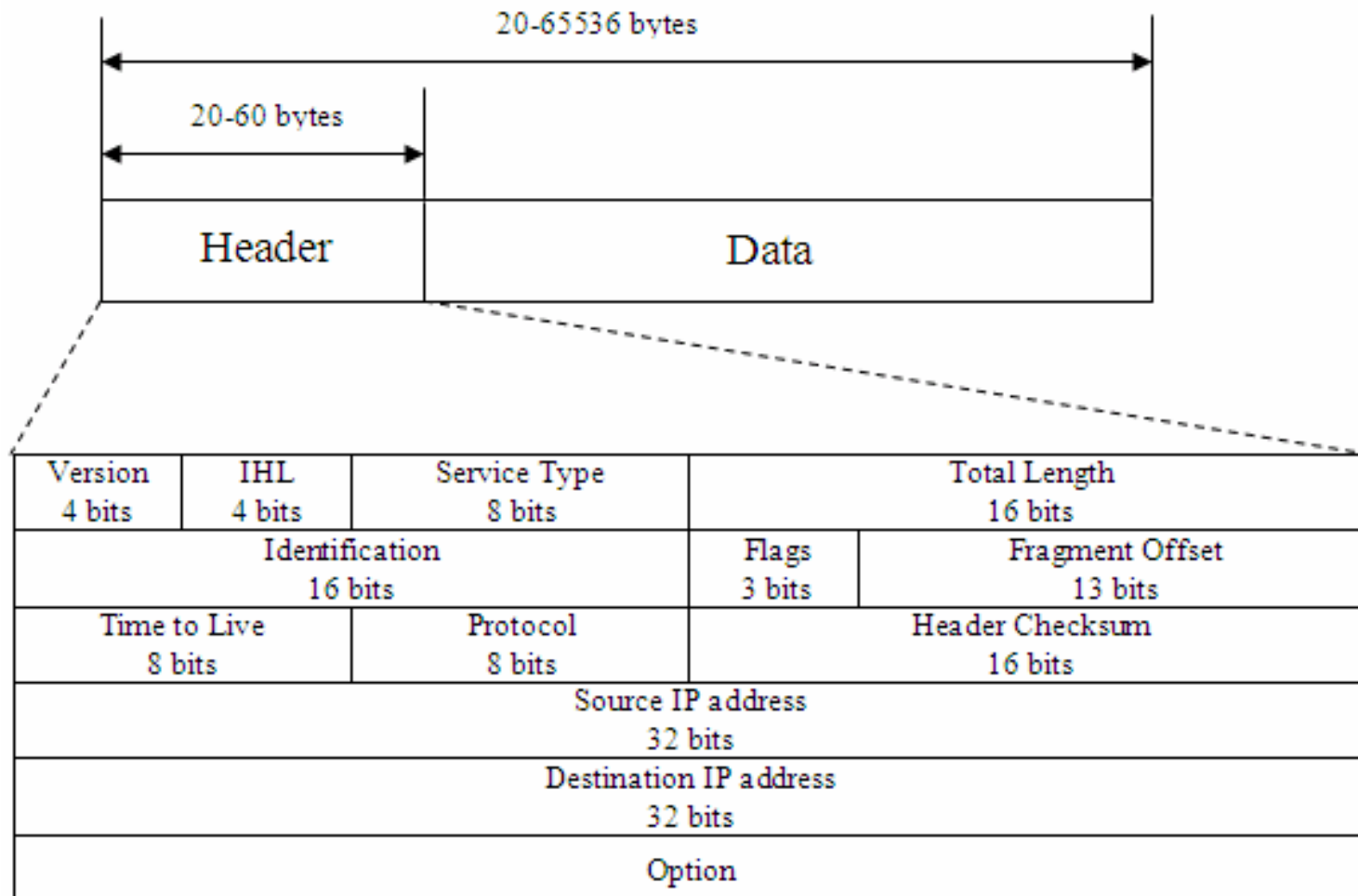
Protocolul IP

Serviciul oferit de protocolul IP

- Serviciul de tipul „best effort” fără conexiune este cel mai simplu serviciu care poate fi oferit în cazul interconectării rețelelor.
- Păstrarea simplității software-ului care rulează pe routere a fost unul din principalele scopuri ale protocolului IP.
- Abilitatea protocolului IP de a rula peste orice tip de rețea reprezintă o altă caracteristică fundamentală a acestuia.
- Multe din tehnologiile peste care protocolul IP rulează nici măcar nu erau inventate în momentul apariției acestuia.

Protocolul IP

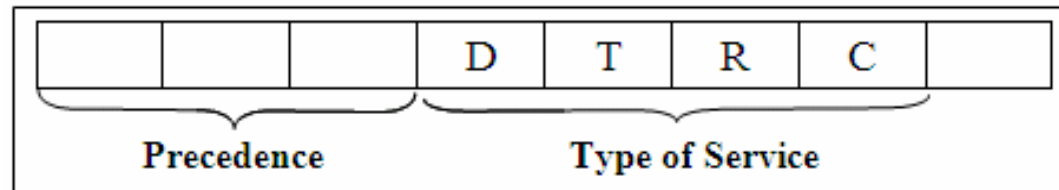
Formatul unui pachet IP



Protocolul IP

Formatul unui pachet IP

- **Version:** Versiunea protocolului. Există două versiuni funcționale, 4 și 6. În acest material este prezentată doar versiunea 4.
- **IHL:** Câmpul ne spune lungimea header-ului, exprimată în număr de cuvinte de 4 octeți. Dacă lungimea este 20, atunci valoarea lui IHL este 5.
- **Service Type:** Câmpul acesta este împărțit la rândul lui în mai multe subcâmpuri.



- **Precedence:** a fost gândit pentru a defini prioritatea unui pachet. În versiunea 4 a protocolului IP acest subcâmp nu este folosit.

Protocolul IP

Formatul unui pachet IP

- **Type of Service:** acest subcâmp este format din 4 biți. Fiecare dintre ei are o anumite semnificație și doar unul poate fi setat la un moment dat.

ToS	Semnificație
0000	Normal
0001	Minimizează costul
0010	Maximizează siguranța
0100	Maximizează capacitatea de transfer
1000	Minimizează întârzierea

- **Total Length:** Acest câmp conține lungimea totală a pachetului. Dacă se dorește să se afle lungimea datelor, se scade din lungimea totală, valoarea câmpului Header Length înmulțită cu 4.
- **Identification, Flags, Fragmentation Offset:** Folosite în procesul de fragmentare a pachetelor.

Protocolul IP

Formatul unui pachet IP

- ***Time to Live:*** Acest câmp este folosit pentru a stabili numărul maxim de hop-uri (routere) prin care un pachet poate trece.
- Fiecare router care procesează pachetul decrementează câmpul cu o unitate. Când valoarea ajunge la zero, pachetul este eliminat din rețea și un mesaj de eroare este generat către nodul care avea adresa trecută în câmpul *Source IP Address*.
- Valoarea de inițializare a acestui câmp este de obicei dublul numărului maxim de router-e care se pot interpune între sursă și destinație.
- Este necesar acest mecanism deoarece în absența lui și în anumite circumstanțe (tabele de rutare corupte) anumite pachete ar putea călători la infinit în rețea, consumând inutil resursele rețelei.

Protocolul IP

Formatul unui pachet IP

- **Protocol:** Prin acest câmp se identifică protocolul de nivel superior care face uz de protocolul IP pentru a-și transporta datele

Protocol	Valoare
TCP	6
UDP	17
ICMP	1
OSPF	89
EGP	8
Ipv6	41

- **Checksum:** Sumă de control aplicată pachetului.
- **Source Address:** Câmpul conține adresa nodului care a trimis pachetul.
- **Destination Address:** Câmpul conține adresa nodului căruia îi este destinat pachetul.

Protocolul IP

Fragmentarea și reasamblarea pachetelor

- Fiecare protocol de pe Nivelul Legătură de Date are propriul format pentru frame-urile utilizate la transportul informației. Când un pachet IP traversează diferite rețelele el trebuie să fie încapsulat în aceste frame-uri ale Nivelului Legătură de Date.
- Fiecare frame acceptă o anumită dimensiune maximă pentru câmpul de date. Astfel, dacă dimensiunea pachetului IP depășește această valoare, pachetul va trebui să fie fragmentat.
- Câmpul *Flags* conține 3 biți. Primul este rezervat, iar următorii doi sunt notați cu D (*do not fragment*), respectiv M(*more fragment*). Dacă bitul D are valoarea 1, atunci pachetul nu poate fi fragmentat. Dacă bitul M are valoarea 1 aceasta semnifică că pachetul nu este ultimul fragment ci mai sunt și altele. Dacă valoarea este 0, atunci fragmentul este ultimul.
 - Câmpul *Fragmentation Offset* indică poziția relativă a unui fragment în cadrul unui pachet. Poziția este indicată sub formă de deplasament exprimat ca multiplu de 8 octeți.

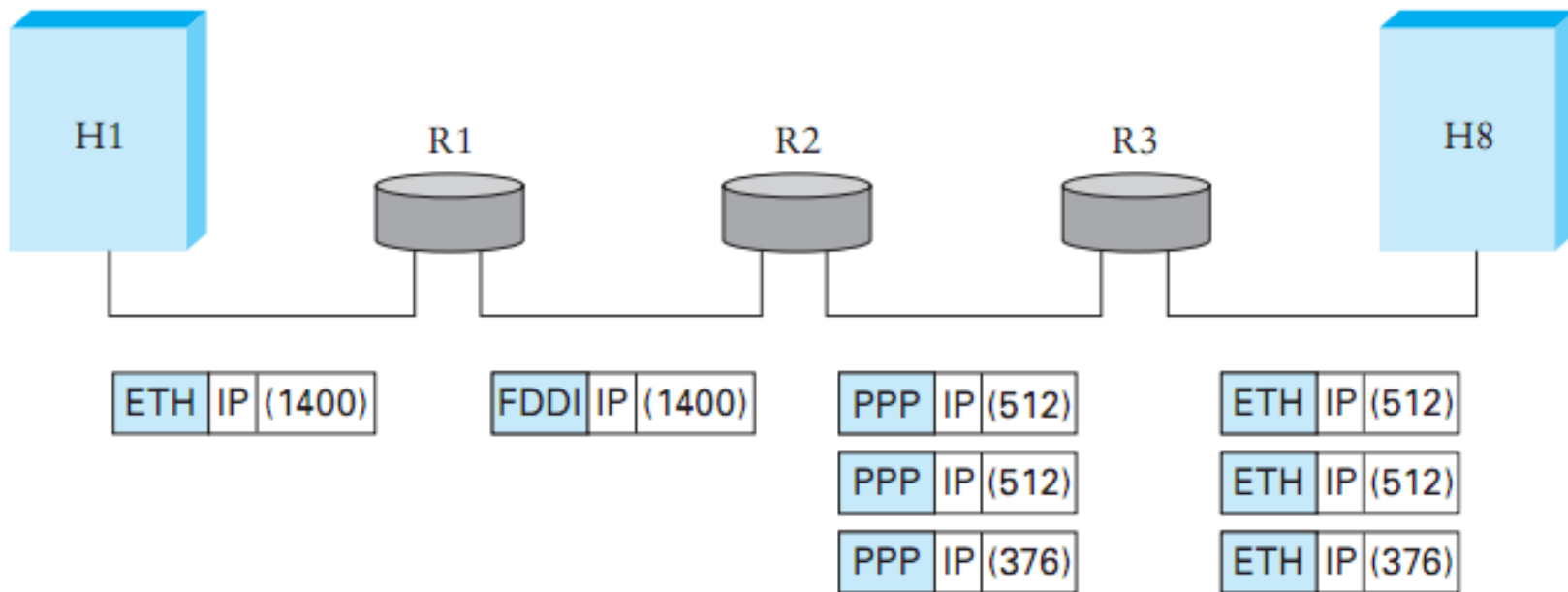
Protocolul IP

Fragmentarea și reasamblarea pachetelor

- Câmpul *Identification* are și el un rol important în procesul de fragmentare. Fiecare pachet primește un număr de identificare care va fi stocat în acest câmp. Acest număr este generat prin incrementare cu unu pentru fiecare nou pachet trimis. Valoarea de la care se pornește este una pozitivă aleasă în mod aleator. Astfel, pachetele vor fi identificate în mod unic folosind această etichetă și adresa sursei. Atunci când este necesar ca un pachet să fie fragmentat, fragmentele care au rezultat vor avea același număr de identificare cu cel al pachetului din care provin. În acest fel va putea fi refăcut pachetul original.

Protocolul IP

Fragmentarea și reasamblarea pachetelor



Protocolul IP

Fragmentarea și reasamblarea pachetelor

(a)

Start of header			
Ident = x		0	Offset = 0
Rest of header			
1400 data bytes			

(b)

Start of header			
Ident = x		1	Offset = 0
Rest of header			
512 data bytes			

Start of header			
Ident = x		1	Offset = 64
Rest of header			
512 data bytes			

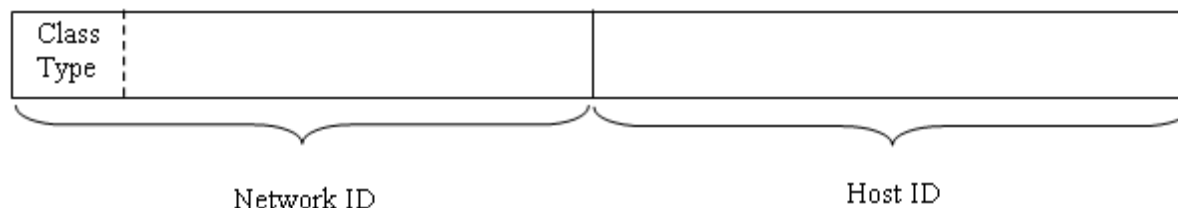
Start of header			
Ident = x		0	Offset = 128
Rest of header			
376 data bytes			

Header fields used in IP fragmentation. (a) Unfragmented packet; (b) fragmented packets.

Protocolul IP

Adresarea

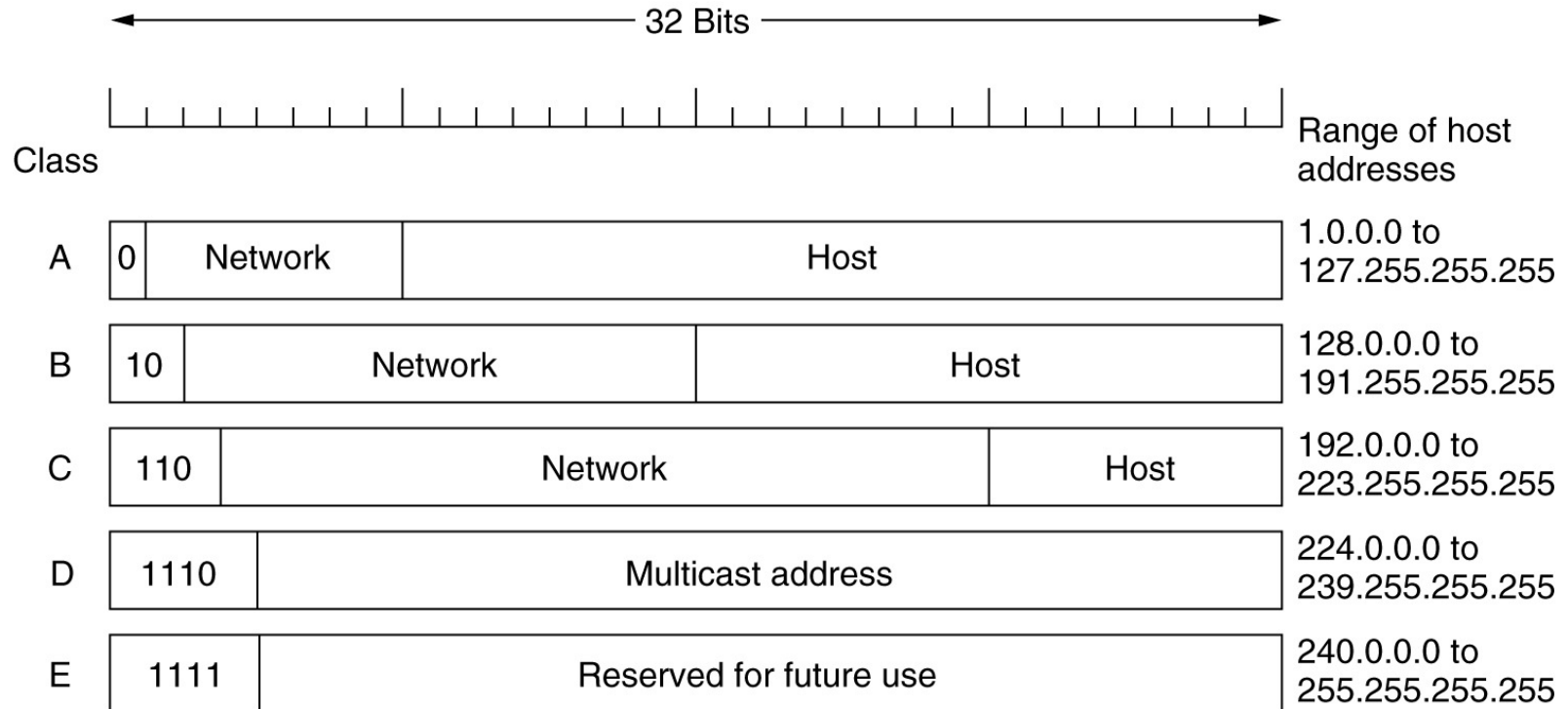
- Protocolul IP implementează o schemă de adresare globală.
- Întregul spațiu de adrese IP a fost împărțit în 5 clase notate de la A la E.
- O adresă IP conține două părți: una care ne dă adresa rețelei și cealaltă ne spune adresa host-ului în cadrul rețelei.
- Valuarea primilor biți dintr-o adresă IP ne spune clasa din care face parte acea adresă. Fiecare clasă alocă un număr diferit de biți pentru partea de Network ID și pentru Host ID.



Class	High Order Bits	Netid	Hostid
A	0	7 bits	24 bits
B	10	14 bits	16 bits
C	110	21 bits	8 bits
D	1110	28 bits multicast groupe number	
E	1111	reserved	

Protocolul IP

Adresarea



Protocolul TCP

- Protocolul TCP are sarcina de a transforma Nivelul Rețea, reprezentat în cazul nostru prin protocolul IP, dintr-un nivel nesigur într-unul sigur.
- Tot protocolul TCP este responsabil și cu implementarea anumitor mecanisme de **control al fluxului și al congestiei**.
- Protocolul **TCP este de tipul capăt la capăt**, adică este necesar să existe o instanță a acestui protocol doar pe mașina sursă și pe mașina destinație, nu și în nodurile intermediare care vor fi tranzitate de către pachete.

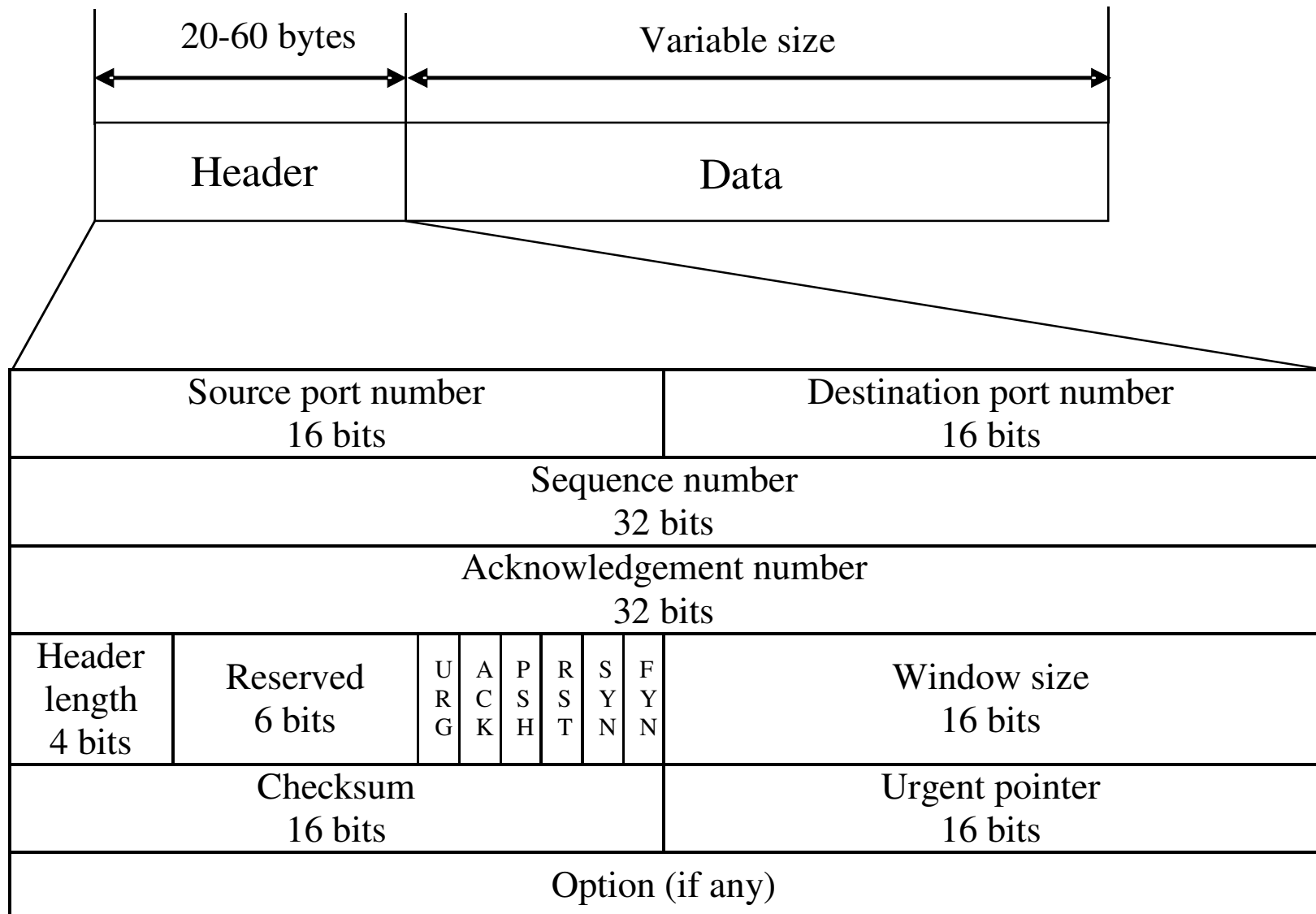
Protocolul TCP

Serviciile oferite de către TCP sunt:

- Stream Data Service
 - Aceasta presupune că pachetele care ajung la destinație să fie recepționate exact în ordinea în care au fost trimise.
 - Dacă trebuie transmis un mesaj, care din cauza lungimii va fi fragmentat în mai multe pachete, la receptor se verifică sosirea tuturor pachetelor și se așează în ordinea în care au fost trimise.
 - Pentru a putea furniza acest serviciu, TCP face uz de buffer-e atât pe partea de transmisie, cât și pe partea de recepție.
- Full-duplex service
 - Presupune că transferul de informații între două noduri poate fi făcut simultan în ambele direcții.
 - Când un pachet pleacă de la unul din noduri către celălalt, transportă și un mesaj de confirmare pentru un pachet recepționat anterior (piggybacking).
- Reliable service
 - TCP folosește un mecanism de confirmări pentru a se asigura că pachetele nu au fost afectate de erori și ca au sosit în ordinea corectă.

Protocolul TCP

Formatul unui pachet TCP



Protocolul TCP

Formatul unui pachet TCP

- **Source port number** : Numărul portului folosit de către aplicația care rulează pe mașina care trimite pachetele.
- **Destination port number** : Numărul portului folosit de aplicația care rulează pe mașina care primește pachetele.
- **Sequence number**: TCP numerotează fiecare octet trimis și folosește acest câmp pentru a indica numărul de secvență al primului octet de date din pachet. Când se inițiază o comunicație între două mașini, numărul de secvență pentru primul octet trimis este ales aleator.
- **Acknowledgement number**: Acest câmp este folosit pentru a confirma octeții recepționați. El se calculează însumând la valoarea din câmpul *Sequence number* al pachetului primit, dimensiunea câmpului de date recepționat. Astfel valoarea acestui câmp reprezintă de fapt următoarea valoare pentru câmpul *Sequence number* care va fi folosită de către mașina care recepționează pachetul de confirmare.

Protocolul TCP

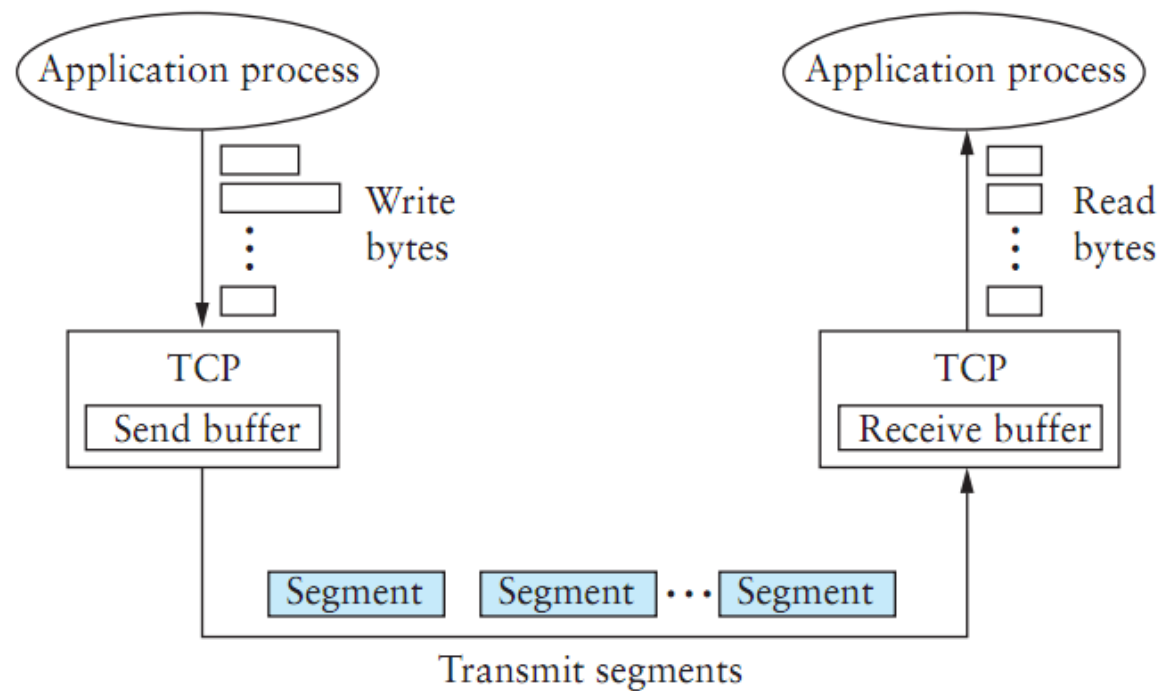
Formatul unui pachet TCP

- **Header length:** Conține dimensiunea header-ului exprimată în cuvinte de 32 de biți.
- **Control field:** Conține 6 biți a căror semnificație va fi explicată atunci când se va discuta modul cum se inițiază și se desfășoară o sesiune de comunicație.
- **Window size:** reprezintă spațiul disponibil din buffer-ul de recepție.

Protocolul TCP

Conexiunea de tip end-to-end

- TCP-ul își bazează funcționarea pe un algoritm cu fereastră glisantă.
- Protocelele cu fereastră glisantă au fost gândite pentru legături punct la punct.
- În cazul TCP-ului se vor stabili conexiuni logice între procese care pot rula pe oricare doua calculatoare din Internet.



Protocolul TCP

Conexiunea de tip end-to-end

- Mecanismul ferestrei glisante garantează un transfer sigur al datelor:
 - mecanism de retransmisie al datelor
 - recepția pachetelor în ordinea în care au fost transmise
 - multiplexarea traficului
 - permite controlul fluxului între transmițător și receptor
- Un alt mecanism esențial implementat de către TCP este controlul congestiei.

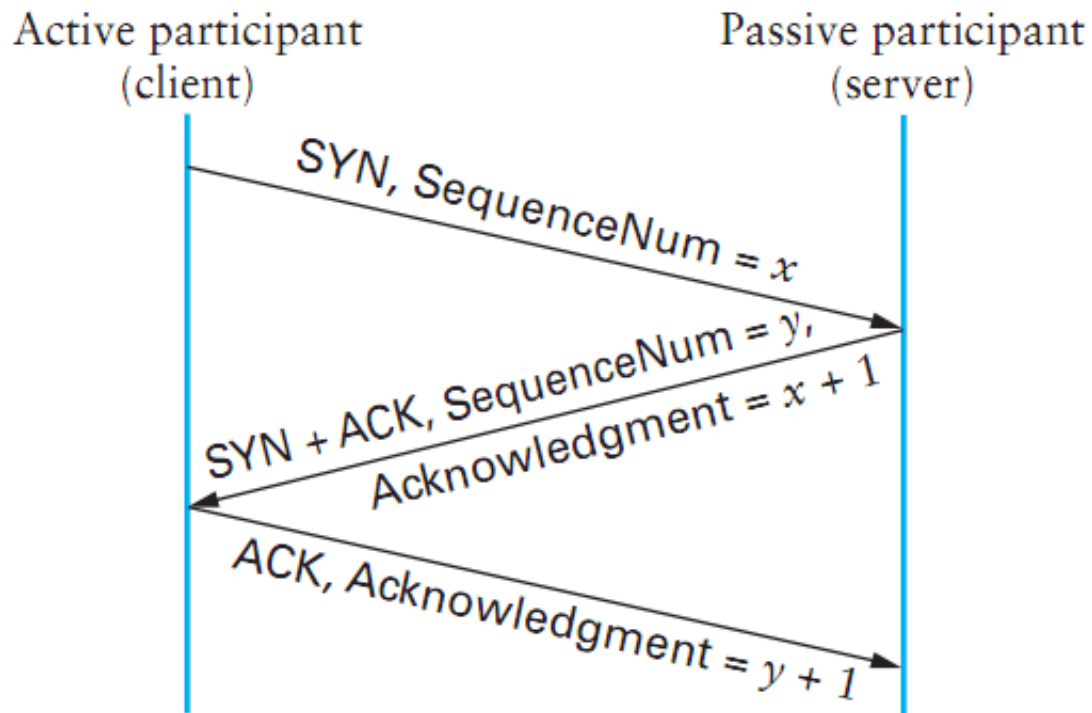
Protocolul TCP

Inițierea conexiunii:

- Nodul care inițiază conexiunea, de obicei este de tip *client*, iar cel care răspunde cererii venite de la *client*, este *server-ul*.
- Pentru a iniția o conexiune este nevoie de trei pachete (*three-way handshake*):
 - Primul pachet vine din partea clientului și are setat flag-ul SYN. Acest pachet conține numărul portului pe care clientul îl va folosi pe durata conexiunii, precum și numărul de secvență inițial.
 - Al doilea pachet implicat în stabilirea conexiunii vine ca răspuns din partea serverului, având setat flag-ul SYN, precum și flag-ul ACK, fiind un pachet de confirmare pentru primul pachet. El mai conține, de asemenea și numărul de secvență inițial folosit de server.
 - Al treilea pachet vine din partea clientului și conține confirmarea pentru pachetul SYN trimis de server.

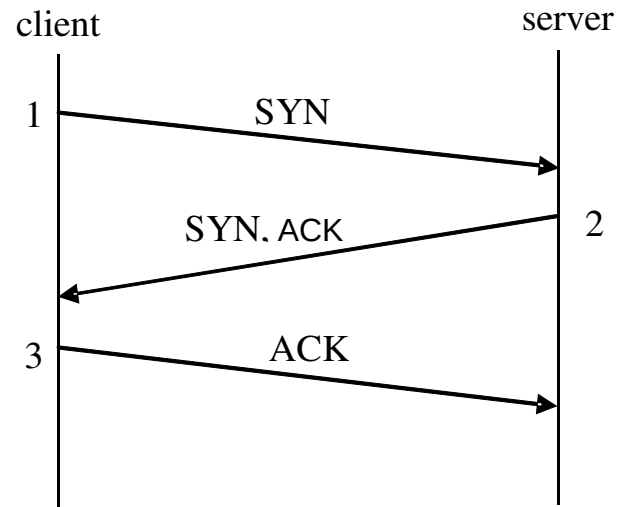
Protocolul TCP

Inițierea conexiunii:



Protocolul TCP

Inițierea conexiunii:

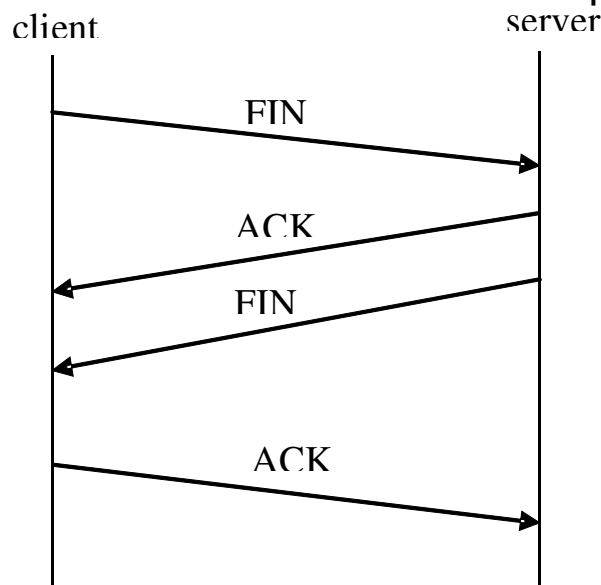


No. ↓	Time	Source	Destination	Protocol	Info
3	0.000309	192.168.0.13	212.112.238.74	TCP	1193 > ftp [SYN] seq=0 Ack=0 win=64240 Len=0 MSS=1460
4	0.131235	212.112.238.74	192.168.0.13	TCP	ftp > 1193 [SYN, ACK] seq=0 Ack=1 win=5840 Len=0 MSS=1460
5	0.131369	192.168.0.13	212.112.238.74	TCP	1193 > ftp [ACK] seq=1 Ack=1 win=64240 Len=0

Protocolul TCP

Încheierea unei conexiuni

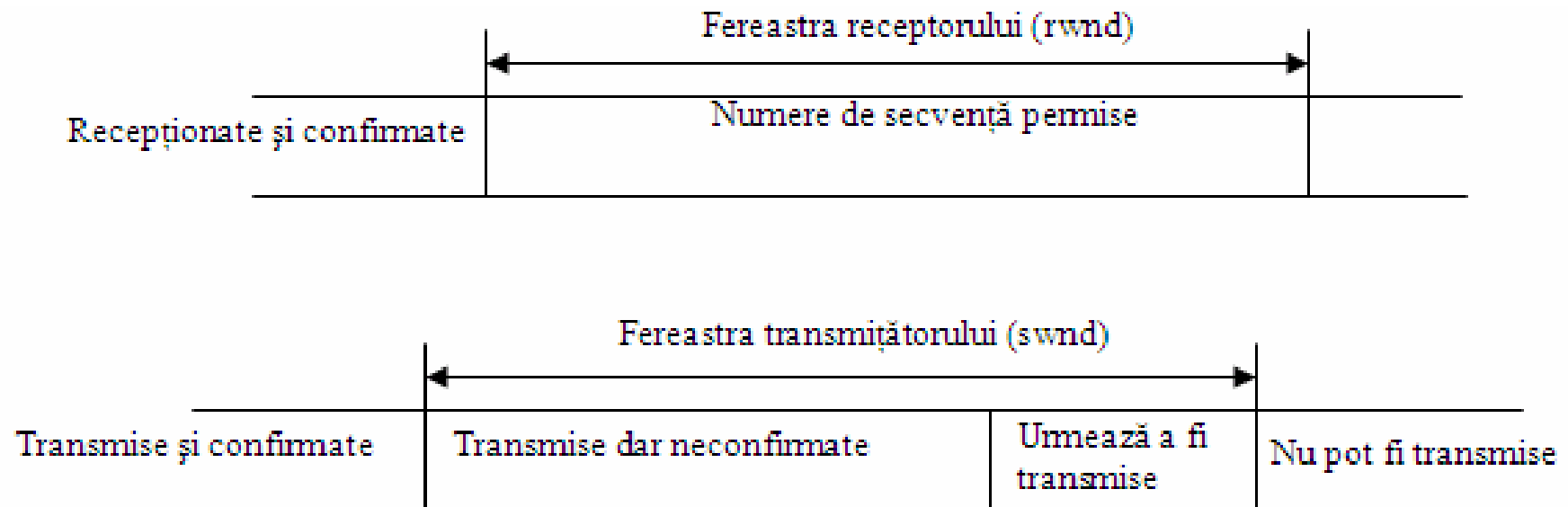
- Încheierea unei conexiuni poate fi făcută în mod bilateral, adică ambele noduri vor trimite câte un pachet de tip FIN, confirmate prin câte un pachet ACK, sau unilateral, când doar unul dintre noduri trimite un pachet FIN care va fi confirmat print-un pachet ACK.



No. ↓	Time	Source	Destination	Protocol	Info
158	24.365722	10.23.3.21	10.23.3.11	TCP	telnet > 1100 [FIN, ACK] Seq=6442 Ack=129 win=5840 Len=0
159	24.366030	10.23.3.11	10.23.3.21	TCP	1100 > telnet [ACK] Seq=129 Ack=6443 win=64171
160	24.366613	10.23.3.11	10.23.3.21	TCP	1100 > telnet [FIN, ACK] Seq=129 Ack=6443 win=64171
161	24.366778	10.23.3.21	10.23.3.11	TCP	telnet > 1100 [ACK] Seq=6443 Ack=130 win=5840 Len=0

Protocolul TCP

Funcționarea ferestrei glisante:



Protocolul TCP

Funcționarea ferestrei glisante:

- Atunci când un pachet este recepționat se verifică dacă numărul lui de secvență coincide cu numărul de secvență de la începutul ferestrei receptorului, adică este următorul număr de secvență așteptat.
- Dacă numărul de secvență nu coincide dar se află în interiorul ferestrei, atunci este introdus în buff-er dar nu este confirmat, iar receptorul trimite un pachet de confirmare care conține în câmpul ACK aceiași valoare cu cea din pachetul de confirmare corespunzător ultimului pachet de date valid.
- Dacă numărul de secvență nu este cel așteptat și nici nu se află în interiorul ferestrei de recepție, atunci pachetul este ignorat.
- În momentul când sosește pachetul așteptat, atunci acesta este confirmat, iar limita din stânga a ferestrei se deplasează spre dreapta.
- Limita din dreapta a ferestrei se va deplasa spre dreapta doar în momentul în care pachetele care au fost confirmate sunt scoase din buffer pentru a fi procesate.

Protocolul TCP

Retransmisia datelor:

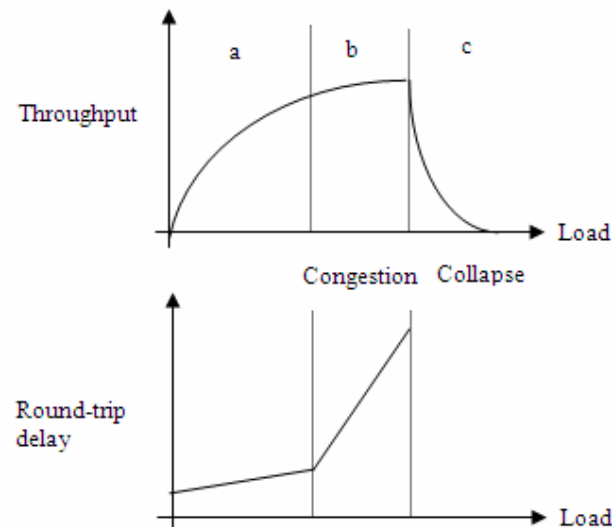
- Retransmisia datelor se realizează fie când pachetele ajung la destinație dar sunt afectate de erori, fie când s-au pierdut pe drum.
- Pentru ca procedeul de retransmisie să funcționeze este nevoie de folosirea unor timere și a unui mecanism de confirmări pozitive (*positive acknowledgements*).
- Confirmarea pozitivă înseamnă că sunt confirmate doar pachetele care au ajuns neafectate de erori.
- Pentru a optimiza mecanismul de retransmisie se folosește o metodă cumulativă de confirmări, care permite confirmarea printr-un singur mesaj a unui grup de pachete consecutive.
- Pentru transmisia confirmărilor sunt folosite pachete de date, procedeul purtând denumirea de *piggybacking*. Există și posibilitatea de a transmite distinct doar pachete de confirmare.

Controlul congestiei

- Atunci când resursele rețelei nu mai reușesc să facă față traficului și rețeaua devine suprasolicitată, parametrii în care se desfășoară traficul se degradează tot mai mult, în felul acesta instalându-se congestia. Congestia se manifestă prin întârzieri tot mai mari, printr-un număr mare de pachete pierdute și în final se poate ajunge la colaps total, adică blocarea rețelei.
- În funcție de locul unde apare congestia există două categorii de congestie:
 - Congestie care apare din cauza suprasolicitării server-elor de aplicații (Server Side Congestion): apar prea multe cereri din partea clienților la un moment dat astfel încât noile cereri pentru conexiuni vor fi respinse.
 - Cealaltă categorie de congestie apare pe partea de client, atunci când mai mulți clienți împart în comun aceleași conexiune fizice. În acest caz congestia apare în nodurile intermediare care nu mai pot să gestioneze numărul mare de conexiuni care apar la un moment dat.
- Într-o rețea bazată pe comutarea de pachete resursele sunt distribuite la nivelul fiecărui nod din rețea. Aceste resurse pot fi definite prin trei elemente: capacitatea de procesare a informațiilor, dimensiunea buffer-elor și capacitatea de transport a liniilor.

Controlul congestiei

- Luând în considerare capacitatea de transport și întârzierile care apar, instalarea congestiei arată grafic ca în figura de mai jos.
- Se observă că atunci când rețeaua lucrează în parametrii optimi, ea răspunde corect atunci când apare o încărcare mai mare (zona a). La început apare o creștere exponențială, apoi o zonă în care rețeaua nu mai reacționează la o creștere suplimentară a încărcării (zona b), pentru ca apoi dintr-un anumit punct capacitatea de transfer să scadă brusc, iar dacă nu sunt luate măsuri, se ajunge la colaps (zona c).
- În ceea ce privește întârzierea introdusă de rețea se observă că se păstrează o valoare aproximativ constantă în zona a, apoi pe măsură ce rețeaua începe să fie congestionată (zona b) întârzierea introdusă de rețea devine tot mai mare.



Controlul congestiei

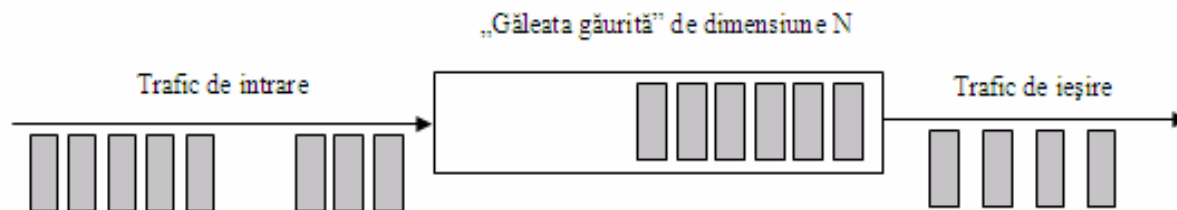
- Pentru controlul congestiei există două abordări și anume:
 - un control în buclă deschisă
 - un control în buclă închisă (cu *feedback*)
- În primul caz se stabilesc de la început parametrii în care va funcționa rețeaua, luându-se măsuri în faza de proiectare pentru prevenirea apariției problemelor, deoarece controlul se va face fără informații despre situația concretă de la un moment dat din rețea. Controlul în buclă deschisă se face de obicei la periferia rețelei, prin supravegherea traficului (*traffic policing*) și prin formarea traficului (*traffic shaping*) pentru nodul care a primit acces la resursele rețelei. Formarea traficului presupune uniformizarea ratei medii de transmisie a datelor.
- Al doilea mod de control al congestiei este cel în buclă închisă. Aici măsurile care sunt întreprinse se bazează pe informații culese în permanență din interiorul rețelei. Informațiile primite pot fi implicite sau explicite. Un tip de informație implicită ar putea fi numărul de pachete pierdute, sau întârzierile din rețea. Informațiile explicite sunt cele generate în mod special pentru a avertiza despre apariția congestiei. Aceste informații pot fi pachete suplimentare care să conțină date despre congestie sau ar putea fi folosite anumite câmpuri în cadrul pachetelor și care vor fi setate cu anumite valori atunci când apare congestia.

Controlul congestiei

- Pentru controlul congestiei în buclă deschisă, vom da două exemple de algoritmi pentru formarea traficului:

Algoritmul găleții găurite

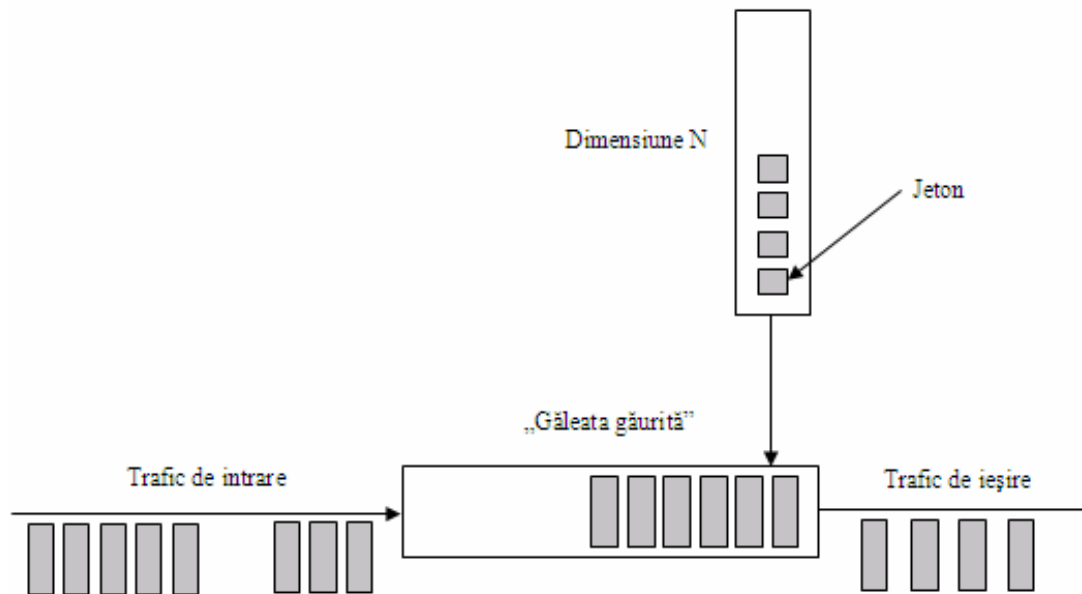
- Algoritmul poartă această denumire deoarece se face analogie cu o găleată care are un orificiu pe fund. Indiferent de debitul cu care apa intră în găleată, ea se va scurge prin orificiu cu un debit constant. La un moment dat, dacă găleata continuă să fie alimentată ea se poate umple și apa se pierde. Același principiu poate fi aplicat și în cazul unei transmisii de date. Pachetele care sunt trimise dintr-un anumit nod în rețea sunt trecute printr-o „găleată găurită”, adică un buffer de tip FIFO care acceptă pachete la orice rată de transfer, dar le transmite mai departe cu o rată fixă. Dacă buffer-ul se umple atunci pachetele care sosesc ulterior se pierd. În felul acesta se realizează în mod implicit și controlul traficului, adică urmărirea dacă un utilizator depășește parametrii de trafic care i-au fost atribuiți. Marele avantaj al acestei metode este acela că nu permite traficului în rafală care ar putea veni din partea nodului transmițător să pătrundă sub această formă în rețea, aceasta fiind principala cauză a apariției congestiei, chiar și atunci când resursele rețelei ar părea ca sunt suficiente.



Controlul congestiei

Algoritmul găleții cu jeton

- Acest algoritm se aseamănă în mare măsură cu cel anterior, dar spre deosebire de acesta permite flexibilitate în ceea ce privește rata traficului de ieșire. În cazul găleții găurite rata acestui trafic era fixă. Algoritmul funcționează în felul următor: găleata acumulează jetoane generate cu o rată de un jeton la ΔT secunde. Pentru ca un pachet să poată fi trimis el trebuie să găsească un jeton în găleată, pe care să-l distrugă. Acest mod de abordare a problemei permite ca în momentul în care la intrare avem date în rafală, iar în găleată avem jetoane disponibile, atunci datele vor fi transmise tot în rafală, dar lungimea rafalei are maxim valoarea egală cu numărul de jetoane din găleată.



Controlul congestiei

Controlul în buclă închisă

- Pentru a se evita funcționarea rețelei în zona de congestie este nevoie să se folosească procedee de monitorizare și control a congestiei. Când se pune problema controlului congestiei două mecanisme trebuie luate în discuție: evitarea congestiei (*congestion avoidance*) și ieșirea din congestie (*congestion recovery*). Aceste mecanisme pot fi implementate pe de o parte la nivelul routerelor, care reprezintă nodurile intermediare, atunci când are loc un transfer de date între sursă și destinație, cât și la nivelul sursei și a destinației, adică al celor două noduri între care se desfășoară transferul. Se spune că în acest caz realizăm un control capăt la capăt.
- Într-o rețea ideală, pentru ca cele două tipuri de control să fie eficiente este nevoie ca pe de o parte rețeaua să ofere feedback, pentru ca resursele să fie folosite în mod eficient, iar pe de altă parte este nevoie ca fluxurile de date să fie protejate unele față de altele, în cazul în care unii utilizatori ar avea tendința să acapareze mai multe resurse decât cele care le-au fost alocate. Această protejare a fluxurilor de date se poate face prin mecanisme de tip QoS.
- Principalul avantaj al unei rețele bazate pe comutarea de pachete este faptul că toate resursele rețelei vor fi folosite împreună de către toți utilizatorii, iar rețeaua va încerca să aloce maximul de resurse disponibile fiecărui utilizator în parte. Ceea ce creează probleme este natura impredictibilă și în rafală a traficului, aceasta putând să conducă la apariția congestiei . Pentru scurte momente de timp rețeaua poate deveni supraîncărcată și pentru a se evita intrarea în congestie este necesar ca într-un anumit fel, utilizatorii să fie înștiințați de acest lucru și să treacă la o diminuare a încărcării rețelei, evitându-se astfel apariția congestiei. Acest mecanism va funcționa, făcând presupunerea că toți utilizatorii rețelei vor coopera și vor lua în considerare semnalele care avertizează asupra apariției congestiei. Pentru a se asigura acest lucru au fost implementate mecanisme de evitare a congestiei chiar în protocoalele de comunicație în Internet.